

Chapitre 2

Méthodologie

La méthodologie adoptée repose sur une démarche structurée pour répondre aux contraintes des réseaux de capteurs multimédias. Elle présente successivement les étapes de conception des trois prototypes d’algorithmes proposés – versions distinctes d’une même approche, différenciées par les méthodes employées à chaque étape (sélection des capteurs, type de données multimédias, méthodes de compression). Elle détaille ensuite les mécanismes de comparaison et d’évaluation mis en œuvre, puis expose les critères de performance retenus : consommation énergétique, qualité des données reconstituées et pertinence des décisions. L’évaluation comparative à travers les tests permettra de désigner la version la plus optimale parmi les trois. Ce cadre cohérent permet de tester, justifier et optimiser les solutions dans des environnements fortement contraints.



2.1 APPROCHES

2.1.1 Algorithme WZ-ASEG : Codage Distribué par Segmentation Hybride et Compression Wyner-Ziv

Contexte et motivation

Les réseaux de capteurs d'images sans fil (WMSN - Wireless Multimedia Sensor Networks) pour l'agriculture de précision se heurtent à une contrainte fondamentale : l'asymétrie énergétique entre les nœuds capteurs alimentés par batterie et les stations de base disposant de ressources quasi-illimitées. Cette asymétrie impose une répartition judicieuse des tâches de traitement d'image entre ces deux entités [1]. Traditionnellement, les approches de compression d'image telles que JPEG ou H.264 concentrent la complexité algorithmique au niveau de l'encodeur, ce qui s'avère inadapté aux contraintes énergétiques des nœuds capteurs [5].

Le codage distribué de sources (DVC - Distributed Video Coding), fondé sur les théorèmes de Slepian-Wolf et Wyner-Ziv [3, 4], offre une alternative prometteuse en inversant ce paradigme : la complexité est transférée au décodeur (le puits) tandis que l'encodeur (le nœud capteur) effectue des opérations simples. Cette architecture asymétrique s'aligne naturellement avec les contraintes des WMSN, où la durée de vie du nœud constitue le facteur limitant principal [6].

Cependant, l'application directe du codage Wyner-Ziv aux images agricoles présente des limites. Les scènes agricoles se caractérisent par une forte corrélation temporelle, une prédominance de zones homogènes (ciel, sol), et des régions d'intérêt clairement définies (végétation). L'algorithme WZ-ASEG (Wyner-Ziv with Otsu-Hue Segmentation) que nous proposons exploite ces propriétés spécifiques en intégrant une segmentation légère basée sur les méthodes d'Otsu [12] et de seuillage colorimétrique, permettant de transmettre uniquement l'information pertinente pour l'analyse agronomique.

L'originalité de cette approche réside dans trois aspects principaux : premièrement, la détection de changements significatifs évite les transmissions redondantes, prolongeant ainsi l'autonomie énergétique ; deuxièmement, la segmentation hybride Otsu-Hue identifie précisément les zones végétales sans calculs coûteux ; troisièmement, l'intégration du codage Wyner-Ziv permet de réduire le débit transmis en exploitant la corrélation temporelle au niveau du décodeur.

Architecture générale de l'algorithme

L'architecture de WZ-ASEG repose sur un modèle client-serveur asymétrique illustré par le logigramme de la figure ???. Le système se décompose en deux entités aux responsabilités distinctes : le nœud capteur (encodeur léger) et le puits (décodeur complexe). Cette séparation fonctionnelle s'appuie sur le principe fondamental du codage distribué, où l'exploitation de la redondance statistique entre images successives est déléguée au décodeur [8].

Au niveau du nœud capteur, l'architecture privilégie la simplicité opérationnelle. L'acquisition d'images à résolution modérée (typiquement 640×480 pixels sur 8 bits) minimise le volume de données à traiter. Un mécanisme de détection de changements basé sur des indicateurs globaux (moyenne, variance de luminance, proportion de pixels verts) permet d'éviter les transmissions inutiles lorsque la scène reste stable. Cette approche s'inspire des travaux sur la détection de mouvement dans les réseaux de capteurs [38], adaptés ici



au contexte statique de la surveillance agricole.

Lorsqu'un changement significatif est détecté, le nœud effectue une segmentation en deux étapes. La première étape applique la méthode d'Otsu [12] sur l'image en niveaux de gris pour discriminer les objets du fond selon leur intensité lumineuse. Cette méthode, qui maximise la variance inter-classe, présente l'avantage de ne nécessiter qu'un parcours de l'histogramme (complexité $O(L)$ où L est le nombre de niveaux de gris) et de ne requérir aucun paramètre utilisateur. La seconde étape exploite l'espace colorimétrique HSV (Hue, Saturation, Value) pour isoler la végétation par seuillage de la teinte. Le canal de teinte (Hue) offre une invariance partielle aux variations d'illumination [13], propriété essentielle pour les applications en extérieur soumises aux cycles jour/nuit et aux conditions météorologiques variables.

La fusion des deux masques de segmentation (Otsu et Hue) par opération logique ET produit une carte binaire robuste des régions d'intérêt. Cette approche hybride combine les avantages de la segmentation par intensité (robustesse au bruit, détection des contours nets) et de la segmentation par teinte (sélectivité pour la végétation). Les opérations morphologiques de fermeture peuvent être appliquées pour éliminer les artefacts isolés et améliorer la connectivité des régions segmentées.

L'encodage Wyner-Ziv constitue la phase finale du traitement au nœud. L'image (ou ses coefficients transformés par DCT) est quantifiée de manière volontairement grossière, puis encodée par un code de canal, typiquement LDPC (Low-Density Parity-Check) ou turbo-code [7]. Contrairement aux codeurs conventionnels, l'encodeur Wyner-Ziv ne génère que des bits de syndrome (parité) sans estimation du mouvement ni prédiction temporelle. Cette simplicité se traduit par une consommation énergétique réduite au prix d'une efficacité de compression moindre que H.264, mais le compromis est favorable dans le contexte énergétiquement contraint des WMSN [6].

Le puits, disposant de ressources computationnelles abondantes, assume la complexité du décodage. Il construit une information auxiliaire (side information) en exploitant les images précédemment reçues, généralement par interpolation temporelle ou prédiction de mouvement [9]. Le décodeur Wyner-Ziv utilise cette side information comme référence approximative de l'image courante et corrige les erreurs de prédiction en exploitant les syndromes reçus via un algorithme de propagation de croyance (belief propagation) dans le graphe de Tanner du code LDPC [10]. Cette approche itérative converge vers une estimation fiable de l'image quantifiée, qui est ensuite déquantifiée et reconstruite.

L'architecture WZ-OSEG tire parti de cette asymétrie naturelle : le nœud effectue $O(n)$ opérations simples (conversions, seuillages, encodage de syndrome) tandis que le puits réalise des traitements complexes (interpolation temporelle, décodage itératif) sans impact sur l'autonomie énergétique du réseau.

Spécification algorithmique

La spécification formelle de l'algorithme WZ-OSEG se décompose en deux modules principaux correspondant aux deux entités du système. Cette section présente le pseudo-code détaillé de chaque module, en explicitant les structures de données, les paramètres de configuration et les dépendances entre fonctions.

Module d'encodage au nœud capteur Le module d'encodage implémente un pipeline de traitement séquentiel visant à minimiser l'empreinte mémoire et la latence de traitement. Le traitement par blocs, initié par la fonction `INIT_TRAITEMENT_PAR_BLOCS`,





permet de traiter l'image par fragments de taille $b \times b$ pixels (typiquement 8×8 ou 16×16), évitant ainsi de charger l'intégralité de l'image en mémoire vive. Cette approche est particulièrement adaptée aux microcontrôleurs disposant de quelques centaines de kilo-octets de RAM.

La détection de changements repose sur un vecteur d'indicateurs $\mathbf{ind} \in R^d$ où d représente le nombre de descripteurs globaux extraits. Ces indicateurs incluent typiquement : la moyenne de luminance μ_L , la variance de luminance σ_L^2 , l'entropie de l'histogramme H , et la proportion de pixels verts p_{green} . La distance entre le vecteur courant $\mathbf{ind}_t$ et le vecteur précédent $\mathbf{ind}_{t-1}$ est calculée via une métrique L_2 normalisée :

$$d(\mathbf{ind}_t, \mathbf{ind}_{t-1}) = \sqrt{\sum_{i=1}^d w_i \left(\frac{\mathbf{ind}_{t,i} - \mathbf{ind}_{t-1,i}}{\sigma_i} \right)^2} \quad (2.1)$$

où w_i sont des poids permettant de pondérer l'importance relative de chaque indicateur, et σ_i les écarts-types normalisateurs. Si $d < \tau_{chg}$ (seuil de détection de changement), le nœud construit un paquet minimal contenant uniquement les métadonnées (timestamp, indicateurs) et le transmet, évitant ainsi l'encodage et la transmission de l'image complète.

En cas de changement détecté, la segmentation hybride Otsu-Hue est activée. La conversion RGB vers niveaux de gris suit la recommandation ITU-R BT.601 [22] :

$$L = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B \quad (2.2)$$

Le seuil d'Otsu θ^* est déterminé en maximisant la variance inter-classe σ_B^2 [12] :

$$\theta^* = \arg \max_{\theta \in [0, 255]} \sigma_B^2(\theta) = \arg \max_{\theta} [\omega_0(\theta) \omega_1(\theta) (\mu_0(\theta) - \mu_1(\theta))^2] \quad (2.3)$$

où ω_0 et ω_1 sont les probabilités cumulées des deux classes, et μ_0 et μ_1 leurs moyennes respectives.

La conversion RGB vers HSV pour le seuillage de teinte suit les équations standards de transformation colorimétrique [14]. Le masque M_{hue} est obtenu en testant l'appartenance de la teinte H à un intervalle $[H_{min}, H_{max}]$ correspondant aux nuances de vert (typiquement $[30, 90]$ pour capturer du vert jaunâtre au vert bleuté). La fusion des masques s'effectue par conjonction logique :

$$M_{ROI}(x, y) = M_{Otsu}(x, y) \wedge M_{Hue}(x, y) \quad (2.4)$$

garantissant qu'un pixel n'est considéré comme région d'intérêt que s'il satisfait simultanément les critères d'intensité et de teinte.

L'encodage Wyner-Ziv appliqué aux coefficients transformés (DCT ou ondelettes) suit le schéma proposé par Aaron et al. [7]. La quantification scalaire uniforme avec pas Δ produit des indices quantifiés $q = \lfloor x/\Delta \rfloor$. Les syndromes sont générés par multiplication matricielle $\mathbf{s} = \mathbf{H} \cdot \mathbf{q} \bmod 2$ où $\mathbf{H}$ est la matrice de parité du code LDPC. La taille du syndrome, proportionnelle au taux de compression souhaité, est adaptée dynamiquement selon le niveau énergétique du nœud.

ALGORITHME: WZ_OSEG_Encodage_Noed

ENTRÉE: ParamWZ (paramètres Wyner-Ziv),

ParamHue (seuils HSV),

Seuil_chg (seuil détection changement)

SORTIE: Transmission paquet compressé ou message minimal





DEBUT

```

t ← HORODATAGE()
I ← ACQUERIR_IMAGE() // Acquisition résolution native
INIT_TRAITEMENT_PAR_BLOCS(I, ParamWZ.taille_bloc)

// Phase 1: Extraction indicateurs et détection changement
ind ← CALCUL_INDICATEURS_GLOBAUX(I)
// ind contient: {_L, 2_L, H_hist, p_green, _R, _G, _B}

d ← DISTANCE_INDICATEURS_L2(ind, EtatMemoire.ind_prev)
// Distance euclidienne normalisée avec pondération

SI d < Seuil_chg ALORS
    // Scène stable, transmission minimale
    msg ← CONSTRUIRE_MESSAGE_MINIMAL(t, ind, "NO_CHANGE")
    TRANSMETTRE(msg) // Paquet < 100 octets
    EtatMemoire.ind_prev ← ind
    EtatMemoire.t_prev ← t
    RETOURNER
FIN SI

// Phase 2: Segmentation hybride Otsu-Hue
Ig ← CONVERTIR_GRIIS(I) // ITU-R BT.601
th ← SEUIL_OTSU(Ig) // Maximisation variance inter-classe
M_otstu ← SEUILLER(Ig, th) // Masque binaire intensité

Ihsv ← CONVERTIR_HSV(I) // Transformation RGB→HSV
M_hue ← SEUILLER_HUE(Ihsv, ParamHue.Hmin, ParamHue.Hmax)
// Extraction pixels verts: H [30°, 90°], S > S_min, V > V_min

M_roi ← FUSION_MASQUES_ET(M_otstu, M_hue)
// Raffinement morphologique optionnel:
// M_roi ← FERMETURE_MORPHO(M_roi, noyau=3×3)

// Phase 3: Transformation et quantification
SI ParamWZ.transform_type = "DCT" ALORS
    X ← DCT_PAR_BLOCS(I, ParamWZ.taille_bloc)
SINON SI ParamWZ.transform_type = "DWT" ALORS
    X ← ONDELETTES_2D(I, ParamWZ.niveau_decomp)
SINON
    X ← SOUS_ECHANTILLONNAGE(I, ParamWZ.facteur_reduc)
FIN SI

Q ← QUANTIFIER_SCALAIRE(X, ParamWZ.quant_step)
// q_i = x_i / , = ParamWZ.quant_step

// Phase 4: Génération syndromes Wyner-Ziv

```





```

S ← GENERER_SYNDROMES_WZ(Q, ParamWZ.code_syndrome)
// S = H · Q mod 2, où H est matrice parité LDPC
// Taille syndrome adaptée au débit cible

// Phase 5: Assemblage paquet et transmission
stats_seg ← EXTRAIRE_STATS_SEGMENTATION(M_roi)
// stats_seg = {surface_roi, centroïde, bbox, moments}

paquet ← ASSEMBLER_PAQUET_WZ(
    S,                // Syndromes LDPC
    ParamWZ,          // Métadonnées encodage
    stats_seg,        // Statistiques segmentation
    ind,              // Indicateurs globaux
    t                 // Timestamp
)

TRANSMETTRE(paquet)

// Mise à jour mémoire locale
EtatMemoire.ind_prev ← ind
EtatMemoire.t_prev ← t
EtatMemoire.M_roi_prev ← COMPRESSER_RLE(M_roi) // Optionnel
FIN

```

La structure `ParamWZ` encapsule les paramètres de configuration du codage Wyner-Ziv : type de transformation (DCT, DWT, sous-échantillonnage), taille de bloc, pas de quantification, matrice de code de canal (LDPC ou turbo), et facteur de compression cible. Cette modularité permet d'adapter dynamiquement la stratégie d'encodage en fonction du budget énergétique résiduel du nœud.

Module de décodage au puits Le module de décodage au puits exploite la puissance de calcul disponible pour reconstruire une image de qualité maximale à partir des syndromes reçus. Le décodage Wyner-Ziv repose sur la construction d'une information auxiliaire (side information) $\hat{I}_{SI}$ qui constitue une prédiction de l'image courante I_t basée sur les images passées $\{I_{t-k}\}_{k=1}^K$ déjà reconstruites.

Plusieurs stratégies de construction de side information sont envisageables [9] :

- **Répétition de trame** : $\hat{I}_{SI} = I_{t-1}$, adaptée aux scènes quasi-statiques typiques de l'agriculture.
- **Interpolation temporelle** : $\hat{I}_{SI} = \frac{1}{2}(I_{t-1} + I_{t+1})$ pour un décodage différé avec accès bidirectionnel.
- **Estimation de mouvement** : application d'algorithmes de flux optique [46] pour prédire I_t à partir de I_{t-1} et du champ de mouvement estimé.
- **Interpolation par réseau neuronal** : utilisation de réseaux convolutionnels pour générer une prédiction de haute qualité [30].

Le choix de la stratégie dépend du compromis entre qualité de reconstruction et latence. Pour les applications agricoles en temps réel, la répétition de trame ou l'interpolation simple s'avèrent généralement suffisantes.

Le décodage LDPC itératif s'effectue par propagation de croyance (belief propagation) dans le graphe de Tanner [10]. À chaque itération i , les rapports de vraisemblance





logarithmiques (LLR - Log-Likelihood Ratios) sont mis à jour en fonction des syndromes reçus et de la side information. Les LLR initiaux $L_0(q_j)$ pour chaque symbole quantifié q_j sont calculés à partir de la side information $\hat{x}_j^{SI}$ en supposant un modèle de canal additif gaussien :

$$L_0(q_j = k) = \log \frac{P(q_j = k | \hat{x}_j^{SI})}{P(q_j = 0 | \hat{x}_j^{SI})} \quad (2.5)$$

où la probabilité $P(q_j = k | \hat{x}_j^{SI})$ dépend de la distance entre $\hat{x}_j^{SI}$ et les intervalles de quantification. Le décodeur itère jusqu'à convergence (satisfaction de toutes les équations de parité) ou atteint un nombre maximal d'itérations I_{max} (typiquement 50 à 100).

ALGORITHME: WZ_OSEG_Decodage_Puits

ENTRÉE: ModeSideInfo (stratégie information auxiliaire)

SORTIE: Image reconstruite I_rec

DEBUT

message ← RECEVOIR()

SI message.type = "minimal" ALORS

// Pas de changement détecté, mise à jour métadonnées

MemoirePuits.ind_prev ← message.ind

MemoirePuits.t_prev ← message.t

RETOURNER NULL // Pas de nouvelle image

FIN SI

// Extraction composantes paquet Wyner-Ziv

bits_WZ ← message.syndromes

ParamWZ ← message.parametres

stats_seg ← message.segmentation // Statistiques ou masque

ind ← message.indicateurs

t ← message.timestamp

// Phase 1: Construction information auxiliaire (side information)

SI ModeSideInfo = "FRAME_REPEAT" ALORS

$\hat{I}_{SI} \leftarrow \text{MemoirePuits.I_prev}$

SINON SI ModeSideInfo = "TEMPORAL_INTERP" ALORS

SI EXISTE(MemoirePuits.I_next) ALORS

$\hat{I}_{SI} \leftarrow \text{INTERPOLER_TEMPOREL}(\$

MemoirePuits.I_prev,

MemoirePuits.I_next

)

SINON

$\hat{I}_{SI} \leftarrow \text{MemoirePuits.I_prev}$

FIN SI

SINON SI ModeSideInfo = "MOTION_COMP" ALORS

MV ← ESTIMER_MOUVEMENT(

MemoirePuits.I_prev,

MemoirePuits.I_prev_prev





```

    )
    Î_SI ← COMPENSER_MOUVEMENT(MemoirePuits.I_prev, MV)
FIN SI

// Phase 2: Transformation side information
SI ParamWZ.transform_type = "DCT" ALORS
    X_SI ← DCT_PAR_BLOCS(Î_SI, ParamWZ.taille_bloc)
SINON SI ParamWZ.transform_type = "DWT" ALORS
    X_SI ← ONDELETTES_2D(Î_SI, ParamWZ.niveau_decomp)
SINON
    X_SI ← SOUS_ECHANTILLONNER(Î_SI, ParamWZ.facteur_reduc)
FIN SI

// Phase 3: Calcul LLR initiaux basés sur side information
LLR_init ← CALCULER_LLR_SIDE_INFO(X_SI, ParamWZ.quant_step)
// Pour chaque coefficient, modèle gaussien:
//  $L(q=k|x) = \log[P(q=k|x)/P(q=0|x)]$ 

// Phase 4: Décodage LDPC itératif
Q ← DECODER_LDPC_ITERATIF(
    bits_WZ,           // Syndromes reçus
    LLR_init,          // LLR initiaux
    ParamWZ.matrice_H, // Matrice parité LDPC
    max_iter=100,      // Nombre max itérations
    critere_arret="SYNDROME_CHECK"
)
// Propagation de croyance dans graphe de Tanner
// Mise à jour messages nœuds variables nœuds contraintes

// Phase 5: Déquantification et reconstruction
X ← DEQUANTIFIER_CENTROID(Q, ParamWZ.quant_step, X_SI)
// Déquantification au centroïde de l'intervalle pondéré par SI

SI ParamWZ.transform_type = "DCT" ALORS
    I_rec ← IDCT_PAR_BLOCS(X, ParamWZ.taille_bloc)
SINON SI ParamWZ.transform_type = "DWT" ALORS
    I_rec ← ONDELETTES_INVERSE_2D(X, ParamWZ.niveau_decomp)
SINON
    I_rec ← SUR_ECHANTILLONNER(X, ParamWZ.facteur_reduc)
FIN SI

// Phase 6: Post-traitement guidé par segmentation
SI stats_seg NULL ALORS
    // Reconstruction masque ROI à partir statistiques
    M_roi_rec ← RECONSTRUIRE_MASQUE(stats_seg, TAILLE(I_rec))

    // Filtrage adaptatif région par région
    I_rec_roi ← FILTRE_BILATERAL(

```





```

        EXTRAIRE_REGION(I_rec, M_roi_rec),
        sigma_spatial=5,
        sigma_range=50
    )
    I_rec_fond ← FILTRE_GAUSSIEN(
        EXTRAIRE_REGION(I_rec, NON(M_roi_rec)),
        sigma=3
    )

    I_rec ← FUSIONNER_REGIONS(I_rec_roi, I_rec_fond, M_roi_rec)
    SINON
        // Post-traitement global
        I_rec ← FILTRE_DEBLOCKING(I_rec, force=0.3)
    FIN SI

    // Phase 7: Stockage et mise à jour mémoire
    STOCKER_IMAGE(I_rec, t, "collection_reconstructions")

    MemoirePuits.I_prev_prev ← MemoirePuits.I_prev
    MemoirePuits.I_prev ← I_rec
    MemoirePuits.ind_prev ← ind
    MemoirePuits.t_prev ← t

    // Optionnel: Calcul métriques qualité si vérité terrain disponible
    SI EXISTE(I_ground_truth) ALORS
        psnr ← CALCULER_PSNR(I_rec, I_ground_truth)
        ssim ← CALCULER_SSIM(I_rec, I_ground_truth)
        JOURNALISER_METRIQUES(t, psnr, ssim)
    FIN SI

    RETOURNER I_rec
FIN

```

Le post-traitement adaptatif exploite les informations de segmentation pour appliquer des filtres différenciés selon les régions. Les zones d'intérêt (végétation) bénéficient d'un filtrage bilatéral [23] préservant les contours, tandis que le fond peut être lissé plus agressivement. Cette approche améliore significativement la qualité perceptuelle sans augmenter le débit de transmission.

La modularité de l'architecture permet d'intégrer facilement des améliorations futures, telles que des schémas de décodage conjoint source-canal [42] ou des méthodes d'apprentissage profond pour la génération de side information [31].

Analyse de complexité algorithmique

L'évaluation rigoureuse de la complexité algorithmique est essentielle pour valider l'adéquation de WZ-OSSEG aux contraintes des systèmes embarqués contraints. Cette section établit les bornes asymptotiques de complexité temporelle et spatiale pour chaque module, en fonction du nombre de pixels $n = H \times W$ de l'image.





Complexité temporelle côté nœud capteur Décomposons le traitement en phases distinctes et analysons la complexité de chacune :

Phase 1 : Calcul des indicateurs globaux

L'extraction des indicateurs $\mathbf{ind} = \{\mu_L, \sigma_L^2, H_{hist}, p_{green}, \dots\}$ nécessite un parcours complet de l'image. Pour chaque pixel (x, y) , un nombre constant c_1 d'opérations arithmétiques est effectué (accès mémoire, conversions, accumulation). Le coût total s'écrit :

$$T_{indicateurs}(n) = c_1 \cdot n = O(n) \quad (2.6)$$

Phase 2 : Détection de changement

La comparaison des vecteurs d'indicateurs via la distance L_2 normalisée implique d soustractions, d multiplications et une racine carrée, où d est la dimension du vecteur (typiquement $d \leq 10$). Cette phase est donc constante :

$$T_{detection}(n) = O(d) = O(1) \quad (2.7)$$

Phase 3 : Segmentation hybride

La conversion RGB→Gris effectue c_2 opérations par pixel (3 multiplications, 2 additions selon ITU-R BT.601), donc :

$$T_{conv_gris}(n) = c_2 \cdot n = O(n) \quad (2.8)$$

Le calcul du seuil d'Otsu parcourt l'histogramme de $L = 256$ niveaux de gris. Pour chaque niveau θ , la variance inter-classe est calculée en temps constant à partir des statistiques cumulées, d'où :

$$T_{otsu}(n) = O(L) + O(n) = O(n) \quad (2.9)$$

puisque L est une constante fixée à 256. Le seuillage proprement dit est linéaire :

$$T_{seuil}(n) = O(n) \quad (2.10)$$

La conversion RGB→HSV implique pour chaque pixel le calcul de maximum, minimum, et divisions conditionnelles, soit c_3 opérations par pixel :

$$T_{conv_hsv}(n) = c_3 \cdot n = O(n) \quad (2.11)$$

Le seuillage de teinte et la fusion des masques (opération ET bit à bit) sont également linéaires :

$$T_{fusion}(n) = O(n) \quad (2.12)$$

La complexité totale de la segmentation est donc :

$$T_{segmentation}(n) = O(n) + O(n) + O(n) + O(n) + O(n) = O(n) \quad (2.13)$$

Phase 4 : Transformation et quantification

Pour la DCT par blocs de taille $b \times b$, la complexité d'une DCT-2D séparable est $O(b^2 \log b)$ par bloc. Avec $N_b = n/b^2$ blocs, le coût total est :

$$T_{dct}(n) = \frac{n}{b^2} \cdot O(b^2 \log b) = O(n \log b) = O(n) \quad (2.14)$$





car b est une constante (typiquement 8 ou 16). Pour les ondelettes, la complexité de la DWT 2D est $O(n)$ [37]. La quantification scalaire uniforme effectue une division et un arrondi par coefficient, soit $O(n)$ au total.

Phase 5 : Encodage Wyner-Ziv

La génération de syndromes $\mathbf{s} = \mathbf{H} \cdot \mathbf{q} \bmod 2$ pour un code LDPC de taux R nécessite $(1 - R) \cdot n$ bits de syndrome. Chaque bit de syndrome est la somme modulo 2 de d_c bits d'information, où d_c est le degré des nœuds de contrainte dans le graphe de Tanner (typiquement $d_c \in [3, 6]$ pour les codes LDPC optimisés). La complexité totale est :

$$T_{\text{encodage}}(n) = O((1 - R) \cdot n \cdot d_c) = O(n) \quad (2.15)$$

puisque R et d_c sont des constantes.

Complexité temporelle totale côté nœud

En agrégeant toutes les phases, nous obtenons :

$$T_{\text{nœud}}(n) = O(n) + O(1) + O(n) + O(n) + O(n) = O(n) \quad (2.16)$$

Cette complexité linéaire est optimale car chaque pixel doit être lu au moins une fois. Le coefficient constant devant n reste modéré grâce à la simplicité des opérations (conversions, seuillages, additions) comparativement aux algorithmes d'estimation de mouvement utilisés dans H.264 (complexité $O(n \cdot w^2)$ où w est la taille de la fenêtre de recherche).

Complexité spatiale côté nœud capteur L'empreinte mémoire constitue souvent le facteur limitant sur les microcontrôleurs embarqués. Analysons les structures de données principales :

Image source

L'image RGB sur 8 bits par canal occupe :

$$M_{\text{image}} = n \times 3 \text{ octets} \quad (2.17)$$

Pour une résolution 640×480 , cela représente $640 \times 480 \times 3 = 921\,600$ octets ≈ 900 Ko.

Buffers intermédiaires

Le traitement par blocs permet de limiter les buffers temporaires. Un seul bloc de taille $b \times b \times 3$ est nécessaire en mémoire à tout instant :

$$M_{\text{bloc}} = b^2 \times 3 \text{ octets} = O(1) \quad (2.18)$$

L'histogramme des niveaux de gris nécessite $L = 256$ compteurs de 32 bits :

$$M_{\text{histogram}} = L \times 4 = 1024 \text{ octets} = O(1) \quad (2.19)$$

Le masque de segmentation binaire peut être compacté à 1 bit par pixel :

$$M_{\text{masque}} = \frac{n}{8} \text{ octets} = O(n) \quad (2.20)$$

Cependant, si seules des statistiques agrégées du masque sont transmises (surface, centroïde, boîte englobante), la mémoire requise devient constante $O(1)$.

Coefficients transformés et quantifiés

Après transformation et quantification, les coefficients occupent :

$$M_{\text{coeffs}} = n \times \log_2(Q_{\text{levels}})/8 \text{ octets} \quad (2.21)$$





où Q_{levels} est le nombre de niveaux de quantification. Pour une quantification sur 4 bits, $M_{coeffs} = n/2$ octets.

Syndromes

Les syndromes LDPC de taux R occupent :

$$M_{syndromes} = (1 - R) \times n/8 \text{ octets} = O(n) \quad (2.22)$$

Complexité spatiale totale côté nœud

En mode de stockage complet du masque :

$$M_{noeud} = O(n) + O(1) + O(n) + O(n) + O(n) = O(n) \quad (2.23)$$

En mode statistiques agrégées uniquement :

$$M_{noeud}^{opt} = O(n) + O(1) = O(n) \quad (2.24)$$

La mémoire est donc dominée par l'image source et les coefficients. Pour réduire davantage l'empreinte, le traitement en ligne (streaming) pixel par pixel ou bloc par bloc peut être implémenté, ramenant la complexité spatiale à $O(b^2)$ au prix d'une complexification du code.

Complexité temporelle côté puits Le puits disposant de ressources abondantes, la complexité y est moins critique. Analysons néanmoins les opérations principales :

Construction de la side information

La répétition de trame (frame repeat) est triviale : $O(1)$ en temps (simple copie de pointeur). L'interpolation temporelle par moyenne de deux trames nécessite :

$$T_{SI_interp}(n) = O(n) \quad (2.25)$$

L'estimation de mouvement par block matching recherche dans une fenêtre $w \times w$ autour de chaque bloc :

$$T_{SI_motion}(n) = \frac{n}{b^2} \times w^2 \times b^2 = O(n \times w^2) \quad (2.26)$$

Pour $w = 16$, cela donne $O(256n)$, ce qui reste acceptable au puits.

Décodage LDPC itératif

Le décodage par propagation de croyance effectue I itérations sur le graphe de Tanner. À chaque itération, chaque nœud de variable (correspondant à un bit d'information) échange des messages avec d_v nœuds de contrainte, et réciproquement. La complexité par itération est :

$$T_{iter_LDPC}(n) = O(n \times d_v \times d_c) \quad (2.27)$$

où d_v est le degré moyen des nœuds de variable (typiquement 2-3) et d_c celui des nœuds de contrainte (3-6). Pour I itérations :

$$T_{LDPC}(n) = I \times O(n \times d_v \times d_c) = O(n) \quad (2.28)$$

si I est borné (typiquement $I \leq 100$, considéré comme constante asymptotique).

Reconstruction inverse

La IDCT par blocs a la même complexité que la DCT directe :



$$T_{idct}(n) = O(n \log b) = O(n) \quad (2.29)$$

Post-traitement

Le filtrage bilatéral avec fenêtre de rayon r effectue pour chaque pixel une convolution spatiale-range :

$$T_{bilateral}(n) = O(n \times r^2) \quad (2.30)$$

Pour r constant (typiquement 3-5), on a $T_{bilateral}(n) = O(n)$.

Complexité temporelle totale côté puits

$$T_{puits}(n) = O(n) + O(n) + O(n) + O(n) = O(n) \quad (2.31)$$

ou $O(n \times w^2)$ si l'estimation de mouvement est activée. Cette complexité supérieure côté puits est acceptable car elle ne grève pas le budget énergétique des nœuds capteurs.

Complexité de communication Le volume de données transmis influence directement la consommation énergétique du module radio, poste dominant dans le bilan énergétique des nœuds [2]. Le paquet WZ-OSeg comprend :

- **Syndromes LDPC** : $(1 - R) \times n$ bits, où R est le taux de compression cible (typiquement $R \in [0.5, 0.8]$)
- **Masque de segmentation** : n bits si transmission complète, ou ≈ 100 octets si statistiques agrégées
- **Indicateurs globaux** : ≈ 50 octets
- **Métadonnées** : ≈ 20 octets (timestamp, identifiants, paramètres)

Le volume total en mode statistiques agrégées s'écrit :

$$V_{comm} = \frac{(1 - R) \times n}{8} + 170 \text{ octets} \quad (2.32)$$

Pour $n = 307\,200$ pixels (640×480) et $R = 0.6$:

$$V_{comm} = \frac{0.4 \times 307\,200}{8} + 170 \approx 15\,360 + 170 = 15\,530 \text{ octets} \approx 15.2 \text{ Ko} \quad (2.33)$$

Cela représente une réduction d'un facteur ≈ 60 par rapport à l'image RGB originale (900 Ko), et d'un facteur ≈ 20 par rapport à une image JPEG de qualité moyenne (≈ 300 Ko). Cette efficacité justifie l'adoption du codage Wyner-Ziv malgré sa complexité de décodage.

Discussion et perspectives

L'algorithme WZ-OSeg présente plusieurs atouts majeurs qui justifient son intérêt dans le contexte des réseaux de capteurs multimédias sans fil pour l'agriculture de précision.

Asymétrie computationnelle exploitée

La conception distribuée permet de limiter drastiquement la charge computationnelle au niveau des nœuds capteurs. Avec une complexité temporelle $O(n)$ et des opérations arithmétiques simples (additions, comparaisons, seuillages), le traitement peut s'exécuter en quelques dizaines de millisecondes sur un microcontrôleur ARM Cortex-M4 cadencé à



100 MHz. À titre comparatif, l'encodage H.264 nécessiterait plusieurs secondes pour la même résolution en raison des opérations d'estimation de mouvement et de prédiction intra-trame [18].

Efficacité énergétique

L'énergie consommée par transmission E_{tx} sur un canal radio d mètres s'exprime selon le modèle de Friis [40] :

$$E_{tx} = (E_{elec} + \epsilon_{amp} \times d^\alpha) \times V_{comm} \quad (2.34)$$

où E_{elec} est l'énergie de l'électronique par bit (≈ 50 nJ/bit), ϵ_{amp} le coefficient d'amplification (≈ 100 pJ/bit/m² pour $\alpha = 2$), et V_{comm} le volume en bits. La réduction du volume transmis d'un facteur 60 se traduit donc par une diminution proportionnelle de l'énergie consommée, prolongeant significativement l'autonomie du nœud.

Segmentation robuste et adaptée au domaine

La combinaison Otsu-Hue offre une détection robuste de la végétation sans nécessiter d'apprentissage supervisé ni de base de données annotées. Le seuil d'Otsu s'adapte automatiquement aux variations d'illumination globale (ombre portée de nuages, transitions jour/nuits), tandis que le seuillage de teinte exploite la signature colorimétrique caractéristique de la chlorophylle. Cette approche présente cependant des limites : sensibilité aux reflets spéculaires sur feuillage mouillé, confusion avec des objets verts artificiels, difficulté à discriminer végétation saine de végétation stressée.

Qualité de reconstruction

La qualité de l'image reconstruite au puits dépend essentiellement de deux facteurs : la corrélation temporelle entre images successives (exploitée via la side information), et le taux de compression choisi. Des expérimentations préliminaires sur séquences agricoles montrent que des PSNR de 28-32 dB sont atteignables pour des taux de compression $R \in [0.5, 0.7]$, ce qui s'avère suffisant pour des tâches de détection de changements ou de comptage grossier. Pour des applications nécessitant une analyse fine (détection de maladies foliaires, estimation de biomasse), des taux de compression plus conservateurs ou des résolutions supérieures peuvent être nécessaires.

Limites et améliorations futures

Plusieurs axes d'amélioration peuvent être envisagés :

1. **Adaptation du taux de compression** : implémenter un mécanisme de retour (feedback) du puits vers le nœud pour ajuster dynamiquement le pas de quantification selon la complexité de la scène et le budget énergétique résiduel.
2. **Segmentation multi-classe** : étendre la segmentation binaire (végétation/fond) à une segmentation multi-classes (sol, végétation saine, végétation stressée, ciel) pour enrichir les métadonnées transmises.
3. **Side information par apprentissage** : utiliser des réseaux neuronaux légers (type MobileNet [34]) au puits pour générer une side information de meilleure qualité, améliorant ainsi le PSNR sans surcoût au nœud.
4. **Codes de canal adaptatifs** : remplacer les codes LDPC fixes par des codes fountain (LT, Raptor) permettant une adaptation automatique au canal sans retransmission [11].
5. **Compression sélective** : ne transmettre en haute qualité que les régions d'intérêt segmentées, et dégrader fortement le fond (ciel, sol), réduisant encore le débit.





Conclusion

L'algorithme WZ-OSEG constitue une solution prometteuse pour la transmission d'images dans les réseaux de capteurs multimédias contraints en énergie. Sa conception asymétrique exploite judicieusement la différence de ressources entre nœuds et puits, en déléguant la complexité de décodage vers l'infrastructure fixe. La segmentation hybride Otsu-Hue offre une détection robuste des zones végétales sans apprentissage supervisé, tandis que l'intégration du codage Wyner-Ziv garantit une transmission efficace grâce à l'exploitation de la corrélation temporelle.

L'analyse de complexité a établi que le traitement au nœud reste linéaire $O(n)$ avec des opérations arithmétiques simples, compatible avec les microcontrôleurs embarqués modernes. Le volume de données transmis est réduit d'un facteur 20 à 60 par rapport aux approches conventionnelles, se traduisant par une prolongation proportionnelle de l'autonomie énergétique.

Les expérimentations sur images réelles, présentées dans les sections ultérieures de ce mémoire, permettront de valider quantitativement ces performances théoriques et de comparer WZ-OSEG avec l'approche alternative développée ci-après, afin de sélectionner la stratégie optimale pour le déploiement opérationnel du système de surveillance agricole.

2.1.2 Algorithme ADRES : Compression Asymétrique par Dégradation Régionale et Encodage Sélectif

Contexte et positionnement

Si l'algorithme WZ-OSEG présenté précédemment exploite la corrélation temporelle via le codage distribué de Wyner-Ziv, il présente néanmoins certaines limitations pour les applications agricoles. En particulier, la complexité du décodage LDPC itératif au puits, bien que déportée hors du nœud capteur, peut poser des problèmes de latence pour les systèmes temps réel nécessitant une reconstruction immédiate des images. De plus, la qualité de reconstruction dépend fortement de la corrélation temporelle : dans les scénarios où la scène évolue rapidement (vent fort agitant le feuillage, variations d'illumination brutales), la side information devient peu fiable, dégradant les performances du décodeur Wyner-Ziv [9].

L'algorithme ADRES (Asymmetric Degradation and Region-based Encoding Scheme) que nous proposons ici adopte une philosophie radicalement différente. Plutôt que d'exploiter la redondance temporelle entre images successives, ADRES se concentre sur la redondance spatiale intra-image en appliquant une dégradation différenciée selon l'importance perceptuelle des régions. Cette approche s'inspire des travaux sur la compression à qualité variable (Variable Quality Compression) [16] et des mécanismes d'attention visuelle dans les systèmes de vision par ordinateur [21].

Le principe fondamental d'ADRES repose sur une observation simple : dans une image agricole typique, la végétation constitue la région d'intérêt (ROI - Region Of Interest) pour l'analyse agronomique, tandis que le fond (ciel, sol, infrastructures) possède une valeur informationnelle moindre. En allouant sélectivement la bande passante disponible en fonction de l'importance sémantique des régions, ADRES maximise la qualité perceptuelle du contenu pertinent tout en minimisant le volume de données transmis [20].

Cette approche s'inscrit dans le paradigme du codage orienté objet (Object-Based Coding), formalisé par la norme MPEG-4 [17], mais simplifié pour s'adapter aux contraintes des systèmes embarqués. Contrairement aux schémas MPEG-4 qui nécessitent une seg-



mentation précise au pixel près et un encodage de contours sophistiqué, ADRES utilise une segmentation grossière par blocs et une transmission compacte du masque binaire, réduisant drastiquement la complexité computationnelle.

L'architecture asymétrique d'ADRES se manifeste à plusieurs niveaux :

- **Simplicité d'encodage** : segmentation légère par seuillage colorimétrique HSV, sous-échantillonnage spatial, quantification scalaire, compression PNG sans perte — toutes opérations réalisables en $O(n)$ avec arithmétique entière.
- **Sophistication du décodage** : interpolation avancée (bicubique, super-résolution par réseau neuronal), filtrage guidé, fusion multi-échelle — techniques computationnellement coûteuses mais améliorant significativement la qualité visuelle.
- **Adaptation énergétique** : sélection dynamique de profils de compression (Qualité vs Économie) selon le niveau de batterie résiduel, permettant un compromis auto-ajusté entre autonomie et fidélité.

Les contributions principales d'ADRES par rapport à WZ-OSEG sont :

1. **Indépendance temporelle** : chaque image est encodée indépendamment sans référence aux trames précédentes, éliminant la propagation d'erreurs et facilitant l'accès aléatoire.
2. **Décodage déterministe** : reconstruction en une seule passe sans itérations, garantissant une latence bornée et prédictible.
3. **Compression sans perte sur données quantifiées** : utilisation de PNG après quantification, offrant un meilleur rapport compression/complexité que les codes de canal LDPC.
4. **Adaptation énergétique explicite** : mécanisme de profils paramétriques permettant un contrôle fin du compromis qualité/débit/autonomie.

Dans les sections suivantes, nous détaillons l'architecture d'ADRES, spécifions l'algorithme sous forme de pseudo-code modulaire, et analysons rigoureusement sa complexité algorithmique pour démontrer son adéquation aux systèmes embarqués contraints.

Logigramme de l'algorithme ADRES

Le logigramme de la figure ?? illustre le flux d'exécution complet d'ADRES, depuis l'acquisition jusqu'à la transmission côté nœud, et de la réception jusqu'à la reconstruction côté puits. La représentation met en évidence la structure modulaire de l'algorithme et les points de décision critiques régissant l'adaptation énergétique.

Côté nœud capteur (encodage)

Le flux d'encodage se décompose en sept phases séquentielles :

1. **Évaluation énergétique** : lecture du niveau de batterie via ADC (Analog-to-Digital Converter) et sélection du profil de compression adapté. Cette décision binaire détermine l'ensemble des paramètres de traitement ultérieurs.
2. **Acquisition d'image** : capture RGB via capteur CMOS (typiquement OV2640, OV5640) à résolution configurable (640×480 à 1280×720 pixels).
3. **Conversion colorimétrique RGB→HSV** : transformation pixel par pixel selon les équations standards [14], permettant l'accès au canal de teinte invariant à l'illumination.
4. **Segmentation par seuillage de teinte** : classification binaire végétation/fond via test d'appartenance de la teinte H à un intervalle paramétrique, et conditions





additionnelles sur saturation S et valeur V pour éliminer les faux positifs (zones grises désaturées).

5. **Extraction régionale** : séparation spatiale de l'image en deux composantes disjointes (ROI et fond) via masquage binaire, préparant la dégradation différenciée.
6. **Sous-échantillonnage différencié** : application de rapports de sous-échantillonnage distincts R_{ROI} et R_{fond} (typiquement $R_{ROI} = 2$, $R_{fond} = 4$), suivie d'une quantification scalaire avec pas Q_{ROI} et Q_{fond} (typiquement $Q_{ROI} = 1.5$, $Q_{fond} = 2$).
7. **Compression et assemblage** : encodage PNG sans perte des données quantifiées (exploitant la redondance spatiale résiduelle), construction du paquet incluant ROI dégradée, fond dégradé, masque binaire et métadonnées, puis transmission radio.

Côté puits (décodage)

Le flux de décodage comprend six phases :

1. **Réception et décompression** : extraction du paquet radio, décodage PNG des composantes ROI et fond, obtention des données quantifiées.
2. **Déquantification** : multiplication par les pas de quantification Q_{ROI} et Q_{fond} pour restaurer les valeurs approximatives originales (reconstruction au centroïde des intervalles de quantification).
3. **Sur-échantillonnage** : interpolation spatiale bicubique ou par réseau neuronal (SRCNN [28], ESRGAN [29]) pour restaurer la résolution native.
4. **Interpolation avancée de la ROI** : application de techniques sophistiquées (filtrage bilatéral, NLM - Non-Local Means [25], super-résolution) spécifiquement sur la zone d'intérêt pour maximiser la qualité perçue.
5. **Fusion régionale** : recombinaison des composantes ROI et fond via le masque binaire transmis, avec lissage optionnel des discontinuités aux frontières (alpha-blending).
6. **Post-traitement global** : application de filtres guidés [24] ou de débruitage par diffusion anisotrope [26] pour harmoniser l'apparence globale et atténuer les artefacts de compression.

La séparation nette entre traitements légers au nœud (conversions, seuillages, sous-échantillonnage) et traitements sophistiqués au puits (interpolations avancées, filtrage guidé) illustre l'exploitation optimale de l'asymétrie ressources du système.

Spécification algorithmique détaillée

Cette section présente l'implémentation formelle d'ADRES sous forme de pseudo-code structuré en fonctions modulaires. L'approche fonctionnelle facilite la compréhension, la vérification et l'implémentation effective sur plateformes embarquées.

Structures de données et paramètres Avant de présenter les algorithmes, définissons les structures de données principales :

STRUCTURE ProfilCompression:

String nom	// "QUALITE" ou "ECONOMIE"
Entier ratio_ROI	// Facteur sous-échantillonnage ROI
Entier ratio_fond	// Facteur sous-échantillonnage fond





```
Réel quant_ROI           // Pas quantification ROI
Réel quant_fond          // Pas quantification fond
Intervalle seuils_teinte // {h_min1, h_max1, h_min2, h_max2}
Réel seuil_saturation    // S_min
Réel seuil_valeur       // V_min
```

STRUCTURE EtatEnergetique:

```
Réel niveau_batterie // Pourcentage [0, 100]
Réel seuil_basculement // Seuil passage Qualité→Économie
Entier compteur_transmissions
Réel energie_residuelle // En Joules
```

STRUCTURE MetadonneesImage:

```
Horodatage timestamp
Dimensions resolution // {largeur, hauteur}
ProfilCompression profil
Statistiques stats_roi // {surface, centroïde, bbox}
Réel checksum // Vérification intégrité
```

Les profils prédéfinis sont :

PROFIL_QUALITE := ProfilCompression(

```
nom = "QUALITE",
ratio_ROI = 2,
ratio_fond = 4,
quant_ROI = 1.5,
quant_fond = 2.0,
seuils_teinte = {30°, 90°, 150°, 180°}, // Vert + cyan
seuil_saturation = 20%,
seuil_valeur = 15%
```

)

PROFIL_ECONOMIE := ProfilCompression(

```
nom = "ECONOMIE",
ratio_ROI = 4,
ratio_fond = 8,
quant_ROI = 2.0,
quant_fond = 3.0,
seuils_teinte = {35°, 85°, 160°, 180°}, // Intervalle plus restreint
seuil_saturation = 25%,
seuil_valeur = 20%
```

)

Le profil Qualité privilégie la fidélité visuelle avec des dégradations modérées, tandis que le profil Économie maximise la réduction de débit au prix d'une perte de détails.

Fonction principale d'encodage au nœud

ALGORITHME: ADRES_Encodage_Noead





ENTRÉE: EtatEnergetique E

SORTIE: PaquetCompresse D

DEBUT

```
// Phase 1: Sélection profil adaptatif
SI E.niveau_batterie > E.seuil_basculement ALORS
    profil ← PROFIL_QUALITE
    JOURNALISER("Profil QUALITE activé, batterie=" + E.niveau_batterie + "%")
SINON
    profil ← PROFIL_ECONOMIE
    JOURNALISER("Profil ECONOMIE activé, batterie=" + E.niveau_batterie + "%")
FIN SI

// Phase 2: Acquisition image
t ← HORODATAGE_HAUTE_PRECISION() // Timestamp microsecondes
I_rgb ← ACQUERIR_IMAGE_RGB()
(H, W, C) ← DIMENSIONS(I_rgb)
ASSERT(C = 3, "Image RGB attendue")

// Phase 3: Conversion colorimétrique RGB→HSV
I_hsv ← CONVERTIR_RGB_VERS_HSV(I_rgb)
// Implémentation détaillée ci-dessous

// Phase 4: Segmentation par teinte
masque ← SEGMENTER_PAR_TEINTE_AVANCE(I_hsv, profil)
// Retourne masque binaire 1=ROI, 0=fond

// Phase 5: Extraction composantes régionales
ROI ← EXTRAIRE_REGION(I_rgb, masque)
FOND ← EXTRAIRE_REGION(I_rgb, INVERSE(masque))

stats_roi ← CALCULER_STATISTIQUES_MASQUE(masque)
// Surface, centroïde, bbox, moments de Hu

// Phase 6: Dégradation différenciée
// Sous-échantillonnage
ROI_reduit ← SOUS_ECHANTILLONNER_BICUBIQUE(ROI, profil.ratio_ROI)
FOND_reduit ← SOUS_ECHANTILLONNER_BICUBIQUE(FOND, profil.ratio_fond)

// Quantification
ROI_quant ← QUANTIFIER_SCALAIRE_UNIFORME(ROI_reduit, profil.quant_ROI)
FOND_quant ← QUANTIFIER_SCALAIRE_UNIFORME(FOND_reduit, profil.quant_fond)

// Phase 7: Compression sans perte et assemblage
D_ROI ← COMPRESSER_PNG(ROI_quant, niveau=9)
// PNG niveau 9 = compression maximale (Deflate)
D_FOND ← COMPRESSER_PNG(FOND_quant, niveau=9)
masque_compact ← COMPRESSER_RLE(masque)
```





```
// Run-Length Encoding pour masque binaire

metadata ← CONSTRUIRE_METADONNEES(t, {H,W}, profil, stats_roi)

D ← ASSEMBLER_PAQUET(
    donnees_roi = D_ROI,
    donnees_fond = D_FOND,
    masque = masque_compact,
    metadonnees = metadata,
    checksum = CRC32(D_ROI + D_FOND + masque_compact)
)

// Journalisation consommation
taille_paquet ← TAILLE_OCTETS(D)
JOURNALISER("Taille paquet: " + taille_paquet + " octets")
JOURNALISER("Taux compression global: " + (H*W*3 / taille_paquet))

RETOURNER D
FIN
```

Fonctions auxiliaires d'encodage Conversion RGB vers HSV avec gestion cas limites

FONCTION: CONVERTIR_RGB_VERS_HSV
 ENTRÉE: Image RGB I (valeurs [0,255])
 SORTIE: Image HSV I_hsv (H[0°,360°], S[0,100], V[0,100])

```
DEBUT
    (H_img, W_img, _) ← DIMENSIONS(I)
    I_hsv ← CREER_IMAGE(H_img, W_img, 3)

    POUR y de 0 à H_img-1 FAIRE
        POUR x de 0 à W_img-1 FAIRE
            // Extraction et normalisation
            r ← I[y,x,0] / 255.0
            g ← I[y,x,1] / 255.0
            b ← I[y,x,2] / 255.0

            // Calcul V (Valeur)
            v_max ← MAX(r, g, b)
            v_min ← MIN(r, g, b)
            delta ← v_max - v_min

            v ← v_max

            // Calcul S (Saturation)
            SI v_max = 0 ALORS
                s ← 0
```





```

SINON
    s ← delta / v_max
FIN SI

// Calcul H (Teinte)
SI delta = 0 ALORS
    h ← 0 // Teinte indéfinie (gris achromatique)
SINON SI v_max = r ALORS
    h ← 60 * (((g - b) / delta) MOD 6)
SINON SI v_max = g ALORS
    h ← 60 * (((b - r) / delta) + 2)
SINON // v_max = b
    h ← 60 * (((r - g) / delta) + 4)
FIN SI

// Normalisation H dans [0°, 360°]
SI h < 0 ALORS
    h ← h + 360
FIN SI

// Stockage
I_hsv[y,x,0] ← h
I_hsv[y,x,1] ← s * 100
I_hsv[y,x,2] ← v * 100
FIN POUR
FIN POUR

RETOURNER I_hsv
FIN
    
```

Segmentation par teinte avec double intervalle

La segmentation doit gérer la circularité de l'espace de teinte ($360^\circ - 0^\circ$). Pour capturer les verts jaunâtres aux verts bleutés, deux intervalles sont souvent nécessaires.

FONCTION: SEGMENTER_PAR_TEINTE_AVANCE

ENTRÉE: Image HSV I_hsv, ProfilCompression profil

SORTIE: Masque binaire M

DEBUT

```

(H_img, W_img, _) ← DIMENSIONS(I_hsv)
M ← CREER_MASQUE_BINAIRE(H_img, W_img)
    
```

```

// Extraction seuils
h_min1 ← profil.seuils_teinte.h_min1
h_max1 ← profil.seuils_teinte.h_max1
h_min2 ← profil.seuils_teinte.h_min2
h_max2 ← profil.seuils_teinte.h_max2
s_min ← profil.seuil_saturation
v_min ← profil.seuil_valeur
    
```





```

compteur_roi ← 0

POUR y de 0 à H_img-1 FAIRE
  POUR x de 0 à W_img-1 FAIRE
    h ← I_hsv[y,x,0]
    s ← I_hsv[y,x,1]
    v ← I_hsv[y,x,2]

    // Test appartenance teinte (double intervalle)
    cond_teinte ← (h h_min1 ET h h_max1) OU
                  (h h_min2 ET h h_max2)

    // Test saturation et valeur minimales
    cond_saturation ← (s s_min)
    cond_valeur ← (v v_min)

    SI cond_teinte ET cond_saturation ET cond_valeur ALORS
      M[y,x] ← 1 // Pixel ROI
      compteur_roi ← compteur_roi + 1
    SINON
      M[y,x] ← 0 // Pixel fond
    FIN SI
  FIN POUR
FIN POUR

// Raffinement morphologique pour éliminer artefacts
M ← EROSION_MORPHOLOGIQUE(M, noyau_carre(3,3))
M ← DILATATION_MORPHOLOGIQUE(M, noyau_carre(5,5))
// Fermeture morphologique: bouche petits trous, lisse contours

proportion_roi ← compteur_roi / (H_img * W_img)
JOURNALISER("Proportion ROI: " + proportion_roi * 100 + "%")

RETOURNER M
FIN

```

Extraction régionale avec gestion pixels transparents

```

FONCTION: EXTRAIRE_REGION
ENTRÉE: Image RGB I, Masque binaire M
SORTIE: Image RGB I_region (pixels non-masqués mis à 0)

DEBUT
  (H, W, C) ← DIMENSIONS(I)
  I_region ← CREER_IMAGE(H, W, C)

  POUR y de 0 à H-1 FAIRE
    POUR x de 0 à W-1 FAIRE

```





```

    SI M[y,x] = 1 ALORS
        // Copie pixel
        I_region[y,x,:] ← I[y,x,:]
    SINON
        // Pixel masqué → noir (ou couleur neutre)
        I_region[y,x,:] ← [0, 0, 0]
    FIN SI
FIN POUR
FIN POUR

RETOURNER I_region
FIN

```

Sous-échantillonnage bicubique

FONCTION: SOUS_ECHANTILLONNER_BICUBIQUE

ENTRÉE: Image I, Facteur ratio R

SORTIE: Image réduite I_reduit

DEBUT

(H, W, C) ← DIMENSIONS(I)

H_new ← ARRONDIR(H / R)

W_new ← ARRONDIR(W / R)

I_reduit ← CREER_IMAGE(H_new, W_new, C)

POUR i de 0 à H_new-1 FAIRE

POUR j de 0 à W_new-1 FAIRE

// Coordonnées dans image source

y_src ← i * R

x_src ← j * R

// Interpolation bicubique 4×4 voisinage

I_reduit[i,j,:] ← INTERPOLER_BICUBIQUE_4x4(I, x_src, y_src)

FIN POUR

FIN POUR

RETOURNER I_reduit

FIN

// Noyau bicubique de Keys (a=-0.5)

FONCTION: INTERPOLER_BICUBIQUE_4x4

ENTRÉE: Image I, Coordonnées réelles (x, y)

SORTIE: Valeur interpolée v

DEBUT

x_floor ← PLANCHER(x)

y_floor ← PLANCHER(y)

dx ← x - x_floor





```

dy ← y - y_floor

// Calcul poids bicubiques selon Keys
w_x ← CALCULER_POIDS_BICUBIQUE(dx)
w_y ← CALCULER_POIDS_BICUBIQUE(dy)

// Convolution séparable 4×4
valeur ← 0
POUR m de -1 à 2 FAIRE
    POUR n de -1 à 2 FAIRE
        x_voisin ← CLAMP(x_floor + n, 0, LARGEUR(I)-1)
        y_voisin ← CLAMP(y_floor + m, 0, HAUTEUR(I)-1)

        poids ← w_x[n+1] * w_y[m+1]
        valeur ← valeur + I[y_voisin, x_voisin, :] * poids
    FIN POUR
FIN POUR

RETOURNER CLAMP(valeur, 0, 255)
FIN
    
```

Quantification scalaire uniforme

```

FONCTION: QUANTIFIER_SCALAIRE_UNIFORME
ENTRÉE: Image I, Pas quantification Q
SORTIE: Image quantifiée I_quant

DEBUT
    I_quant ← ARRONDIR(I / Q)
    // Quantification uniforme par arrondi au plus proche
    // Nombre de niveaux = 256 / Q

    RETOURNER I_quant
FIN
    
```

La quantification réduit la plage dynamique, facilitant la compression PNG ultérieure en réduisant l'entropie des données.

Fonction principale de décodage au puits

```

ALGORITHME: ADRES_Decodage_Puits
ENTRÉE: PaquetCompresse D
SORTIE: Image reconstruite I_rec

DEBUT
    // Phase 1: Extraction et décompression
    D_ROI ← EXTRAIRE_COMPOSANTE(D, "donnees_roi")
    D_FOND ← EXTRAIRE_COMPOSANTE(D, "donnees_fond")
    masque_compact ← EXTRAIRE_COMPOSANTE(D, "masque")
    
```





```

metadata ← EXTRAIRE_COMPOSANTE(D, "metadonnees")

// Vérification intégrité
checksum_calcule ← CRC32(D_ROI + D_FOND + masque_compact)
ASSERT(checksum_calcule = D.checksum, "Erreur transmission détectée")

// Décompression PNG
ROI_quant ← DECOMPRESSER_PNG(D_ROI)
FOND_quant ← DECOMPRESSER_PNG(D_FOND)
masque ← DECOMPRESSER_RLE(masque_compact)

profil ← metadata.profil
(H_orig, W_orig) ← metadata.resolution

// Phase 2: Déquantification
ROI_dequant ← DEQUANTIFIER(ROI_quant, profil.quant_ROI)
FOND_dequant ← DEQUANTIFIER(FOND_quant, profil.quant_fond)

// Phase 3: Sur-échantillonnage
SI DISPONIBLE(RESEAU_NEURONAL_SUPER_RESOLUTION) ALORS
    // Super-résolution par apprentissage profond
    ROI_restaura ← SUPER_RESOLUTION_SRCNN(
        ROI_dequant,
        facteur=profil.ratio_ROI,
        modele="SRCNN_agricole_pretrained.h5"
    )
    FOND_restaura ← SUR_ECHANTILLONNER_BICUBIQUE(
        FOND_dequant,
        profil.ratio_fond
    )
SINON
    // Interpolation classique si pas de GPU disponible
    ROI_restaura ← SUR_ECHANTILLONNER_BICUBIQUE(
        ROI_dequant,
        profil.ratio_ROI
    )
    FOND_restaura ← SUR_ECHANTILLONNER_BICUBIQUE(
        FOND_dequant,
        profil.ratio_fond
    )
FIN SI

// Phase 4: Interpolation avancée de la ROI
ROI_ameliore ← INTERPOLER_ROI_AVANCEE(ROI_restaura, masque)
// Détail ci-dessous

// Phase 5: Fusion régionale avec lissage frontières
I_fusionnee ← FUSIONNER_REGIONS_ALPHA_BLENDING(

```





```

        ROI_ameliore,
        FOND_restaura,
        masque,
        rayon_feathering=5
    )

// Phase 6: Post-traitement global
I_rec ← FILTRE_GUIDE_EDGE_PRESERVING(
    I_fusionnee,
    guide=I_fusionnee,
    rayon=5,
    epsilon=0.01
)

// Optionnel: débruitage supplémentaire
SI profil.nom = "ECONOMIE" ALORS
    // Compensation dégradation importante en mode Économie
    I_rec ← DEBRUITAGE_NON_LOCAL_MEANS(I_rec, h=10, template_size=7)
FIN SI

// Redimensionnement final si nécessaire
(H_rec, W_rec, _) ← DIMENSIONS(I_rec)
SI H_rec  H_orig OU W_rec  W_orig ALORS
    I_rec ← REDIMENSIONNER(I_rec, H_orig, W_orig, methode="BICUBIQUE")
FIN SI

// Journalisation métriques
JOURNALISER("Reconstruction effectuée, profil=" + profil.nom)
JOURNALISER("Résolution finale: " + H_orig + "x" + W_orig)

// Stockage dans base de données
SAUEVEGARDER_IMAGE(I_rec, metadata.timestamp, "collection_ADRES")

RETOURNER I_rec
FIN
    
```

Fonctions auxiliaires de décodage Déquantification au centroïde

FONCTION: DEQUANTIFIER

ENTRÉE: Image quantifiée I_quant, Pas quantification Q

SORTIE: Image déquantifiée I_dequant

DEBUT

 // Reconstruction au centroïde de l'intervalle de quantification

 I_dequant ← I_quant * Q

 // Ou reconstruction stochastique (dithering) pour réduire banding:

 // I_dequant ← I_quant * Q + UNIFORME(-Q/2, Q/2)





```
// Clamping valeurs dans plage valide
I_dequant ← CLAMP(I_dequant, 0, 255)

RETOURNER I_dequant
FIN
```

Sur-échantillonnage bicubique (opération inverse)

```
FONCTION: SUR_ECHANTILLONNER_BICUBIQUE
ENTRÉE: Image réduite I_reduit, Facteur ratio R
SORTIE: Image agrandie I_agrandi

DEBUT
    (H_reduit, W_reduit, C) ← DIMENSIONS(I_reduit)
    H_agrandi ← H_reduit * R
    W_agrandi ← W_reduit * R

    I_agrandi ← CREER_IMAGE(H_agrandi, W_agrandi, C)

    POUR i de 0 à H_agrandi-1 FAIRE
        POUR j de 0 à W_agrandi-1 FAIRE
            // Mapping vers coordonnées image source
            y_src ← i / R
            x_src ← j / R

            // Interpolation bicubique
            I_agrandi[i,j,:] ← INTERPOLER_BICUBIQUE_4x4(I_reduit, x_src, y_src)
        FIN POUR
    FIN POUR

    RETOURNER I_agrandi
FIN
```

Interpolation avancée de la ROI

Cette fonction applique des techniques sophistiquées pour améliorer la qualité de la région d'intérêt reconstruite.

```
FONCTION: INTERPOLER_ROI_AVANCEE
ENTRÉE: Image ROI_restaura, Masque binaire M
SORTIE: Image ROI_ameliore

DEBUT
    // Extraction zone ROI uniquement
    ROI_seule ← EXTRAIRE_REGION(ROI_restaura, M)

    // Détection contours pour guidance
    contours ← DETECTER_CONTOURS_CANNY(ROI_seule, seuil_bas=50, seuil_haut=150)
```





```
// Filtrage bilatéral préservant contours
ROI_filtree ← FILTRE_BILATERAL(
    ROI_seule,
    sigma_spatial=5,
    sigma_range=50,
    diametre=9
)
// sigma_spatial: lissage spatial
// sigma_range: préservation différences intensité

// Renforcement netteté zone végétation
ROI_nette ← RENFORCER_NETTETE_USM(
    ROI_filtree,
    rayon=1.5,
    quantite=0.8,
    seuil=2
)
// Unsharp Masking pour accentuation détails

// Masquage sélectif: n'appliquer renforcement que sur zones riches en texture
carte_texture ← CALCULER_VARIANCE_LOCALE(ROI_filtree, fenetre=5)
alpha_nettete ← NORMALISER(carte_texture, 0, 1)

ROI_ameliore ← alpha_nettete * ROI_nette + (1 - alpha_nettete) * ROI_filtree

RETOURNER ROI_ameliore
FIN
```

Fusion avec alpha blending aux frontières

FONCTION: FUSIONNER_REGIONS_ALPHA_BLENDING

ENTRÉE: Image ROI, Image FOND, Masque M, Entier rayon_feathering

SORTIE: Image fusionnée I_fusion

DEBUT

(H, W, C) ← DIMENSIONS(ROI)

I_fusion ← CREER_IMAGE(H, W, C)

// Création masque gradué par distance à la frontière

M_distance ← TRANSFORMATION_DISTANCE_EUCLIDIENNE(M)

// M_distance[i,j] = distance au pixel de frontière le plus proche

// Normalisation dans [0,1] sur rayon de feathering

alpha ← CLAMP(M_distance / rayon_feathering, 0, 1)

// Lissage gaussien du masque alpha pour transition douce

alpha ← FILTRE_GAUSSIEN(alpha, sigma=rayon_feathering/3)

// Fusion pondérée pixel par pixel





```

POUR i de 0 à H-1 FAIRE
  POUR j de 0 à W-1 FAIRE
    SI M[i,j] = 1 ALORS
      // Zone intérieure ROI: alpha proche de 1
      I_fusion[i,j,:] ← alpha[i,j] * ROI[i,j,:] +
                        (1 - alpha[i,j]) * FOND[i,j,:]
    SINON
      // Zone fond: utiliser fond directement
      I_fusion[i,j,:] ← FOND[i,j,:]
    FIN SI
  FIN POUR
FIN POUR

RETOURNER I_fusion
FIN

```

Filtre guidé préservant les contours

Le filtre guidé [24] est un filtre de lissage edge-preserving plus rapide que le filtre bilatéral, avec complexité $O(n)$ indépendante du rayon.

FONCTION: FILTRE_GUIDE_EDGE_PRESERVING

ENTRÉE: Image I, Image guide, Entier rayon r, Réel epsilon

SORTIE: Image filtrée I_filtre

DEBUT

```

// Moyennes locales via box filter rapide
mean_I ← BOX_FILTER(I, rayon=r)
mean_guide ← BOX_FILTER(guide, rayon=r)
corr_I_guide ← BOX_FILTER(I * guide, rayon=r)

// Variance locale du guide
var_guide ← BOX_FILTER(guide * guide, rayon=r) - mean_guide * mean_guide

// Coefficients linéaires a et b
a ← (corr_I_guide - mean_I * mean_guide) / (var_guide + )
b ← mean_I - a * mean_guide

// Moyenne des coefficients
mean_a ← BOX_FILTER(a, rayon=r)
mean_b ← BOX_FILTER(b, rayon=r)

// Application filtre
I_filtre ← mean_a * guide + mean_b

```

RETOURNER I_filtre

FIN

Le paramètre ϵ contrôle le degré de lissage : ϵ faible préserve mieux les contours mais lisse moins, ϵ élevé lisse davantage au risque de flouter les détails.





Super-résolution par réseau neuronal convolutionnel

Si le puits dispose d'un GPU, l'utilisation de réseaux neuronaux profonds pour la super-résolution améliore significativement la qualité [28, 29].

FONCTION: SUPER_RESOLUTION_SRCNN

ENTRÉE: Image basse résolution I_lr, Entier facteur, Chemin modele

SORTIE: Image haute résolution I_hr

DEBUT

```
// Chargement modèle pré-entraîné
reseau ← CHARGER_MODELE_KERAS(modele)

// Pré-sur-échantillonnage bicubique (SRCNN fonctionne sur images HR)
I_bicubic ← SUR_ECHANTILLONNER_BICUBIQUE(I_lr, facteur)

// Normalisation [0,1]
I_norm ← I_bicubic / 255.0

// Ajout dimension batch
I_batch ← EXPAND_DIMS(I_norm, axis=0)

// Inférence réseau
I_hr_norm ← reseau.PREDIRE(I_batch)[0]

// Dénormalisation
I_hr ← I_hr_norm * 255.0
I_hr ← CLAMP(I_hr, 0, 255)

RETOURNER I_hr
```

FIN

L'architecture SRCNN typique comprend trois couches convolutionnelles :

- Extraction de patches : Conv(9×9, 64 filtres, ReLU)
- Mapping non-linéaire : Conv(1×1, 32 filtres, ReLU)
- Reconstruction : Conv(5×5, 3 filtres)

Le modèle peut être pré-entraîné sur un dataset d'images agricoles pour spécialisation au domaine.

Analyse de complexité algorithmique

L'analyse de complexité pour ADRES suit une méthodologie similaire à celle appliquée pour WZ-OSeg, mais révèle des différences notables dues à l'absence de codage de canal itératif.

Complexité temporelle côté nœud capteur Soit $n = H \times W$ le nombre de pixels, $C = 3$ le nombre de canaux RGB.

La phase de compression PNG mérite une attention particulière. L'algorithme Deflate utilisé par PNG combine LZ77 (recherche de motifs répétés) et codage de Huffman [19]. La





TABLE 2.1 – Complexité temporelle des phases d'encodage ADRES

Phase	Opération principale	Complexité
1. Évaluation énergétique	Lecture ADC + condition	$O(1)$
2. Acquisition	Lecture n pixels RGB	$O(n)$
3. Conversion RGB→HSV	c_1 ops par pixel	$O(n)$
4. Segmentation teinte	Comparaisons par pixel	$O(n)$
5. Extraction régions	Masquage binaire	$O(n)$
6. Sous-échantillonnage	Interpolation bicubique	$O(n)$
7. Quantification	Division + arrondi	$O(n/R^2)$
8. Compression PNG	Deflate sur n/R^2 pixels	$O((n/R^2) \log(n/R^2))$

complexité théorique de Deflate est $O(m \log m)$ où m est la taille des données en entrée. Pour ADRES :

$$T_{PNG}(n) = O\left(\frac{n}{R^2} \log \frac{n}{R^2}\right) \quad (2.35)$$

Cependant, pour des images quantifiées (faible entropie), les implémentations optimisées de PNG (libpng, stb_image_write) opèrent en pratique proche de $O(n/R^2)$ grâce aux heuristiques de recherche de motifs.

La complexité temporelle totale au nœud s'écrit :

$$T_{noeud}^{ADRES}(n) = O(n) + O(n) + O(n) + O(n) + O\left(\frac{n}{R^2} \log \frac{n}{R^2}\right) \quad (2.36)$$

En pratique, avec $R \in [2, 8]$, le terme logarithmique reste négligeable devant les termes linéaires. On peut donc approximer :

$$T_{noeud}^{ADRES}(n) = O(n) \quad (2.37)$$

Comparaison avec WZ-SEEG : ADRES évite le coût de génération de syndromes LDPC, mais introduit la compression PNG. Le bilan dépend de l'implémentation, mais typiquement PNG sur données quantifiées s'avère plus rapide que l'encodage LDPC itératif.

Complexité spatiale côté nœud capteur Les structures de données en mémoire sont :

TABLE 2.2 – Empreinte mémoire ADRES au nœud

Structure	Taille	Permanence
Image RGB source	$n \times 3$ octets	Transitoire
Image HSV	$n \times 3 \times 4$ octets (float32)	Transitoire
Masque binaire	$n/8$ octets (1 bit/pixel)	Transitoire
ROI sous-échantillonnée	$n/(R_{ROI}^2) \times 3$ octets	Transitoire
Fond sous-échantillonné	$n/(R_{fond}^2) \times 3$ octets	Transitoire
Buffers PNG	$\approx n/R^2$ octets (pire cas)	Transitoire
Métadonnées	≈ 200 octets	Permanent

Le pic mémoire survient lors de la conversion RGB→HSV si les deux images coexistent :



$$M_{noeud}^{pic} = n \times 3 + n \times 12 = 15n \text{ octets} \quad (2.38)$$

Pour $n = 307\,200$ (640×480), cela donne ≈ 4.6 Mo. Cette empreinte peut être réduite en effectuant la conversion en place ou par blocs, ramenant la complexité spatiale à $O(n)$ avec un coefficient plus faible.

En mode optimisé avec traitement streaming :

$$M_{noeud}^{opt} = n \times 3 + \frac{n}{R^2} \times 3 \approx n \times 3 \left(1 + \frac{1}{R^2}\right) = O(n) \quad (2.39)$$

Pour $R = 4$, $M_{noeud}^{opt} \approx 3.2n$ octets ≈ 980 Ko pour 640×480 , compatible avec les microcontrôleurs STM32H7 (1-2 Mo RAM).

Complexité temporelle côté puits Les opérations au puits sont significativement plus coûteuses qu'au nœud, exploitant la puissance de calcul disponible.

TABLE 2.3 – Complexité temporelle des phases de décodage ADRES

Phase	Opération principale	Complexité
1. Décompression PNG	Décodage Deflate	$O((n/R^2) \log(n/R^2))$
2. Déquantification	Multiplication scalaire	$O(n/R^2)$
3. Sur-échantillonnage bicubique	Interpolation 4×4 par pixel	$O(n)$
4. Super-résolution SRCNN	Convolutions 9×9 , 1×1 , 5×5	$O(n \times K)$
5. Filtrage bilatéral ROI	Convolution gaussienne 2D	$O(n_{ROI} \times r^2)$
6. Fusion alpha blending	Pondération linéaire	$O(n)$
7. Filtre guidé	Box filters séparables	$O(n)$

où K est le nombre de filtres convolutionnels du réseau (typiquement $K \approx 100$), et r le rayon du filtre bilatéral.

La complexité totale dépend de l'activation de la super-résolution :

Sans super-résolution :

$$T_{puits}^{ADRES}(n) = O\left(\frac{n}{R^2} \log \frac{n}{R^2}\right) + O(n) + O(n \times r^2) + O(n) = O(n) \quad (2.40)$$

car r est constant (typiquement $r \in [3, 7]$).

Avec super-résolution neuronale :

$$T_{puits}^{ADRES+SR}(n) = O(n \times K) \quad (2.41)$$

Pour SRCNN avec $K = 101$ filtres et $n = 307\,200$ pixels, cela représente environ 31 millions d'opérations MAC (Multiply-Accumulate). Sur GPU (NVIDIA RTX 3060, ≈ 13 TFLOPS), cela s'exécute en quelques millisecondes. Sur CPU (Intel i7-10700, ≈ 500 GFLOPS), cela prend quelques dizaines de millisecondes, acceptable pour une application non temps-réel.

Complexité de communication Le volume transmis par ADRES comprend :

- **ROI compressée** : $V_{ROI} = \frac{n_{ROI}}{R_{ROI}^2 \times Q_{ROI}} \times \rho_{PNG}$ octets
- **Fond compressé** : $V_{fond} = \frac{n_{fond}}{R_{fond}^2 \times Q_{fond}} \times \rho_{PNG}$ octets



— **Masque RLE** : $V_{masque} \approx 0.01n$ octets (compression RLE efficace sur masques binaires)

— **Métadonnées** : $V_{meta} \approx 200$ octets

où ρ_{PNG} est le taux de compression PNG sur données quantifiées (typiquement $\rho_{PNG} \in [0.4, 0.7]$ selon l'entropie résiduelle).

En supposant $n_{ROI} \approx 0.3n$ (30

Profil Qualité ($R_{ROI} = 2$, $R_{fond} = 4$, $Q_{ROI} = 1.5$, $Q_{fond} = 2$, $\rho_{PNG} = 0.6$) :

$$V_{comm}^{qual} = \frac{0.3n}{4 \times 1.5} \times 0.6 + \frac{0.7n}{16 \times 2} \times 0.6 + 0.01n + 200 \quad (2.42)$$

$$= 0.03n + 0.013n + 0.01n + 200 \quad (2.43)$$

$$= 0.053n + 200 \text{ octets} \quad (2.44)$$

Pour $n = 307\,200$: $V_{comm}^{qual} \approx 16\,500$ octets ≈ 16 Ko.

Profil Économie ($R_{ROI} = 4$, $R_{fond} = 8$, $Q_{ROI} = 2$, $Q_{fond} = 3$, $\rho_{PNG} = 0.5$) :

$$V_{comm}^{eco} = \frac{0.3n}{16 \times 2} \times 0.5 + \frac{0.7n}{64 \times 3} \times 0.5 + 0.01n + 200 \quad (2.45)$$

$$= 0.0047n + 0.0018n + 0.01n + 200 \quad (2.46)$$

$$= 0.0165n + 200 \text{ octets} \quad (2.47)$$

Pour $n = 307\,200$: $V_{comm}^{eco} \approx 5\,300$ octets ≈ 5.2 Ko.

Le profil Économie réduit le volume d'un facteur ≈ 3 par rapport au profil Qualité, démontrant l'efficacité de l'adaptation énergétique.

Comparaison avec WZ-OSEG : Pour un taux de compression équivalent, ADRES transmet légèrement plus de données (masque RLE vs statistiques compactes), mais évite le surcoût des syndromes LDPC. En pratique, ADRES et WZ-OSEG atteignent des débits comparables pour des qualités similaires.

Discussion et perspectives

L'algorithme ADRES présente plusieurs avantages distinctifs qui le positionnent comme une alternative crédible à WZ-OSEG dans certains contextes applicatifs.

Simplicité et déterminisme

L'absence de codage itératif rend ADRES plus prédictible en termes de latence et de consommation énergétique. Le nœud effectue un nombre fixe d'opérations indépendamment du contenu de l'image, facilitant la planification temps-réel et l'estimation de la durée de vie de la batterie. Cette propriété est cruciale pour les applications critiques nécessitant des garanties temporelles strictes.

Indépendance temporelle

Contrairement à WZ-OSEG qui exploite la corrélation entre images successives, ADRES encode chaque image indépendamment. Cette indépendance présente plusieurs avantages :

- **Robustesse aux pertes** : la perte d'un paquet ne compromet pas le décodage des paquets suivants, évitant la propagation d'erreurs.
- **Accès aléatoire** : possibilité de reconstruire n'importe quelle image sans dépendance aux trames précédentes, facilitant l'indexation et la recherche dans les archives.





- **Adaptation aux scènes dynamiques** : performances stables même lorsque la corrélation temporelle est faible (vent fort, passages d'animaux).

Qualité perceptuelle optimisée

L'allocation différenciée de la bande passante selon l'importance sémantique des régions améliore la qualité perceptuelle globale. Des études en psychophysique visuelle montrent que les observateurs humains tolèrent mieux une dégradation du fond que de la région d'intérêt [20]. ADRES exploite directement cette propriété.

De plus, l'utilisation de techniques de super-résolution au puits (lorsqu'un GPU est disponible) permet d'atteindre des PSNR comparables à des débits inférieurs. Les réseaux neuronaux pré-entraînés sur des datasets agricoles (PlantVillage [44], Agriculture-Vision [45]) apprennent les prior statistiques des textures végétales, générant des détails plausibles absents de l'interpolation bicubique classique.

Adaptation énergétique dynamique

Le mécanisme de profils paramétriques offre un contrôle fin du compromis qualité/autonomie. En mode Qualité, le système privilégie la fidélité pour les tâches d'analyse critique (détection précoce de maladies, estimation précise de biomasse). En mode Économie, il maximise la durée de vie pour la surveillance de routine. La transition entre profils peut être automatisée via un contrôleur adaptatif basé sur le niveau de batterie et la fréquence de transmission [39].

Limitations et axes d'amélioration

Plusieurs limitations d'ADRES méritent d'être soulignées :

1. **Segmentation binaire rigide** : la classification végétation/fond peut échouer en présence de végétation stressée (feuillage jaunissant, brunissant) dont la teinte sort de l'intervalle prédéfini. Une segmentation multi-classes ou adaptative améliorerait la robustesse.
2. **Artefacts de blocage** : le sous-échantillonnage agressif du fond peut introduire des artefacts visibles (effet de pixelisation). Le sur-échantillonnage bicubique lisse ces artefacts mais ne restaure pas les détails perdus. Des techniques de régularisation (TV - Total Variation [27]) pourraient atténuer ces artefacts.
3. **Dépendance aux conditions d'illumination** : bien que le canal de teinte HSV soit partiellement invariant à l'intensité lumineuse, les seuils de saturation et valeur restent sensibles aux variations d'illumination (ombres portées, nuages). Une normalisation automatique ou un apprentissage des seuils par clustering non supervisé (k-means sur l'espace HSV) améliorerait la robustesse.
4. **Transmission du masque** : même compressé par RLE, le masque binaire représente environ 1-2
5. **Absence d'exploitation de la corrélation temporelle** : dans les scénarios où la scène évolue lentement (croissance végétale sur plusieurs jours), ADRES ne tire pas parti de cette redondance temporelle. Une extension hybride combinant ADRES pour la détection de changements et WZ-OSeg pour l'encodage différentiel pourrait offrir le meilleur des deux approches.

Pistes d'amélioration future

Plusieurs extensions peuvent être envisagées pour améliorer ADRES :

1. **Segmentation sémantique par apprentissage profond** : remplacer le seuillage HSV par un réseau de segmentation léger (U-Net mobile [32], DeepLabV3+ avec





backbone MobileNetV2 [33]) entraîné sur des images agricoles annotées. Cette approche permettrait une segmentation multi-classes (sol, végétation saine, végétation stressée, mauvaises herbes, ciel, infrastructures) avec une robustesse accrue aux variations d'apparence. Le surcoût computationnel au nœud (quelques centaines de millisecondes sur ARM Cortex-A53 avec accélération NEON) reste acceptable pour des applications à fréquence d'acquisition modérée (1 image/minute).

2. **Codage orienté objet avec contours** : transmettre explicitement les contours des régions d'intérêt via des polygones simplifiés (algorithme de Douglas-Peucker [35]) plutôt qu'un masque bitmap. Cette représentation vectorielle s'avère plus compacte pour des ROI aux frontières lisses et permet une reconstruction à résolution arbitraire au puits.
3. **Quantification adaptative basée sur l'activité locale** : au lieu d'une quantification uniforme par région, adapter le pas de quantification selon la complexité locale mesurée par la variance spatiale ou l'énergie des gradients. Les zones riches en textures (feuillage dense) bénéficieraient d'une quantification fine, tandis que les zones homogènes (ciel, sol nu) accepteraient une quantification grossière. Cette approche, inspirée du codage JPEG [15], optimise l'allocation de bits selon l'importance perceptuelle.
4. **Profils adaptatifs multiples** : étendre le système binaire Qualité/Économie à une gradation continue avec N profils (typiquement $N = 5$) offrant un spectre de compromis qualité/débit. Un contrôleur adaptatif basé sur la théorie du contrôle optimal [41] ajusterait dynamiquement le profil selon l'état de charge de la batterie, la fréquence de transmission requise et la complexité de la scène courante.
5. **Transmission progressive par raffinements successifs** : structurer les données en couches hiérarchiques (base layer + enhancement layers) permettant une transmission progressive. Le nœud transmet d'abord une version basse qualité, puis des raffinements successifs si le budget énergétique le permet. Le puits peut interrompre la transmission dès qu'une qualité suffisante est atteinte, économisant de l'énergie. Cette approche s'inspire du codage progressif JPEG2000 [16].
6. **Méta-apprentissage pour adaptation de domaine** : entraîner un méta-modèle (MAML - Model-Agnostic Meta-Learning [43]) capable de s'adapter rapidement aux spécificités d'un nouveau site de déploiement (conditions climatiques, types de cultures, caractéristiques du sol) avec quelques dizaines d'images labellisées seulement. Cette capacité d'adaptation rapide faciliterait le déploiement à large échelle du système ADRES sur des sites hétérogènes.

Conclusion partielle

L'algorithme ADRES (Asymmetric Degradation and Region-based Encoding Scheme) constitue une approche originale pour la compression d'images dans les réseaux de capteurs multimédias contraints en énergie, offrant une alternative complémentaire à WZ-OSeg. Son architecture asymétrique repose sur un encodage simple au nœud (segmentation colorimétrique, sous-échantillonnage différencié, compression PNG) et un décodage sophistiqué au puits (super-résolution, filtrage avancé), exploitant judicieusement la disparité de ressources entre les deux entités.

Les contributions principales d'ADRES sont :





- Une complexité temporelle linéaire $O(n)$ au nœud avec des opérations arithmétiques entières simples, compatible avec les microcontrôleurs embarqués à ressources limitées.
- Une indépendance temporelle garantissant la robustesse aux pertes de paquets et facilitant l'accès aléatoire aux images archivées.
- Un mécanisme d'adaptation énergétique dynamique via profils paramétriques, permettant un compromis auto-ajusté entre qualité de reconstruction et autonomie du nœud.
- Une allocation différenciée de la bande passante selon l'importance sémantique des régions, optimisant la qualité perceptuelle pour les applications d'analyse agronomique.

L'analyse de complexité a établi que ADRES atteint des volumes de transmission de 5 à 16 Ko pour une image 640×480 pixels selon le profil activé, représentant des facteurs de réduction de 60 à 180 par rapport à l'image RGB brute (900 Ko). Cette efficacité se traduit par une prolongation significative de l'autonomie énergétique, facteur critique pour les déploiements longue durée en agriculture de précision.

La comparaison quantitative avec WZ-OSEG, présentée dans les sections expérimentales ultérieures de ce mémoire, permettra de caractériser finement les compromis respectifs des deux approches en termes de qualité de reconstruction (PSNR, SSIM, métriques perceptuelles), débit de transmission, latence de traitement et consommation énergétique. Cette évaluation comparative guidera le choix de l'algorithme optimal selon les contraintes spécifiques de l'application cible (surveillance temps-réel vs archivage haute qualité, scènes statiques vs dynamiques, disponibilité GPU au puits).

2.1.3 Synthèse comparative des deux approches

Les deux algorithmes proposés, WZ-OSEG (Wyner-Ziv with Otsu-Hue Segmentation) et ADRES (Asymmetric Degradation and Region-based Encoding Scheme), incarnent deux paradigmes distincts de compression asymétrique pour réseaux de capteurs multimédias. Cette section établit une synthèse comparative structurée mettant en lumière leurs différences fondamentales, leurs avantages respectifs et leurs domaines d'application privilégiés.

Tableau comparatif des caractéristiques

Analyse des compromis fondamentaux

Compromis efficacité de compression vs complexité

WZ-OSEG atteint potentiellement une meilleure efficacité de compression en exploitant la redondance temporelle, réduisant ainsi le volume transmis. Cependant, cette efficacité se paie au prix d'une complexité accrue au décodage (algorithme de propagation de croyance dans le graphe LDPC, convergence itérative). ADRES privilégie la simplicité (compression PNG standard, décodage en une passe) au prix d'un volume légèrement supérieur dans les scénarios à forte corrélation temporelle.

Compromis robustesse vs efficacité

L'indépendance temporelle d'ADRES garantit qu'une perte de paquet n'affecte qu'une seule image, tandis que dans WZ-OSEG, la perte d'une image dégrade la side information pour les images suivantes, propageant l'erreur. Dans les environnements radio hostiles





TABLE 2.4 – Comparaison synthétique WZ-OSEG vs ADRES

Critère	WZ-OSEG	ADRES
Paradigme de compression	Codage distribué exploitant corrélation temporelle	Codage spatial par dégradation régionale différenciée
Dépendance temporelle	Forte (nécessite side information)	Nulle (encodage indépendant par image)
Complexité encodage nœud	$O(n)$ avec génération syndromes LDPC	$O(n)$ avec compression PNG
Complexité décodage puits	$O(n \times I)$ avec décodage LDPC itératif (I itérations)	$O(n)$ ou $O(n \times K)$ avec super-résolution neuronale
Déterminisme latence	Non (nombre d'itérations variable)	Oui (une seule passe de décodage)
Robustesse pertes paquets	Faible (propagation d'erreurs)	Forte (indépendance temporelle)
Adaptation énergétique	Implicite (via taux de compression)	Explicite (profils paramétriques)
Volume transmission typique	15-20 Ko (640×480 , $R = 0.6$)	5-16 Ko (640×480 , profil dépendant)
Qualité reconstruction	Dépend fortement de la corrélation temporelle	Stable, indépendante du contexte temporel
Segmentation	Otsu + Hue (statistiques compactes)	Seuillage HSV (masque complet ou statistiques)
Post-traitement puits	Filtrage guidé, interpolation temporelle	Super-résolution, filtrage bilatéral, alpha-blending
Complexité implémentation	Élevée (codes LDPC, décodage itératif)	Modérée (sous-échantillonnage, PNG standard)
Scénarios favorables	Scènes statiques ou quasi-statiques, haute corrélation temporelle	Scènes dynamiques, accès aléatoire requis, garanties temps-réel

(obstacles, interférences, mobilité), ADRES s'avère plus résilient. Inversement, dans des canaux fiables, WZ-OSEG maximise l'efficacité spectrale.

Compromis adaptabilité vs prédictibilité

WZ-OSEG s'adapte implicitement à la complexité de la scène via le nombre d'itérations LDPC nécessaires à la convergence, mais cette adaptabilité rend la latence et la consommation énergétique imprévisibles. ADRES offre un contrôle explicite via les profils paramétriques, garantissant des bornes déterministes sur la latence et l'énergie consommée, facilitant la planification dans les systèmes temps-réel.

Domaines d'application privilégiés

WZ-OSEG : surveillance continue de scènes statiques

WZ-OSEG excelle dans les applications de surveillance continue à long terme de parcelles agricoles où la scène évolue lentement (croissance végétale, variations d'illumination graduelles). Exemples typiques :

- Monitoring phénologique (suivi de la croissance végétative sur des semaines/mois)





- Détection d'anomalies à évolution lente (stress hydrique progressif, carences nutritionnelles)
- Systèmes d'irrigation automatique nécessitant une acquisition fréquente (toutes les 15-30 minutes) avec forte corrélation temporelle

ADRES : détection d'événements et analyse ponctuelle

ADRES convient mieux aux applications nécessitant un accès aléatoire aux images, des garanties temps-réel ou opérant dans des environnements dynamiques :

- Détection d'intrusions (animaux nuisibles) avec acquisition déclenchée par capteurs de mouvement
- Inspection périodique de la santé des cultures (1-2 fois par jour) avec analyse immédiate requise
- Archivage d'images haute qualité pour consultation ultérieure sans dépendances temporelles
- Systèmes multi-capteurs avec synchronisation lâche (pas d'ordre temporel strict)

Critères de sélection pour le déploiement

Le choix entre WZ-OSEG et ADRES doit être guidé par une analyse multicritère des contraintes applicatives :

1. **Dynamique de la scène** : Si corrélation temporelle forte (> 0.8) $\rightarrow$ WZ-OSEG. Si scène dynamique ou acquisition sporadique $\rightarrow$ ADRES.
2. **Contraintes temps-réel** : Si latence bornée requise $\rightarrow$ ADRES. Si latence variable acceptable $\rightarrow$ WZ-OSEG.
3. **Fiabilité du canal radio** : Si taux d'erreur paquet $< 1\%$ $\rightarrow$ WZ-OSEG. Si $> 5\%$ $\rightarrow$ ADRES.
4. **Ressources au puits** : Si GPU disponible $\rightarrow$ ADRES avec super-résolution. Si CPU uniquement $\rightarrow$ WZ-OSEG (décodage LDPC optimisé CPU).
5. **Budget énergétique** : Si batterie abondante (solaire, secteur) $\rightarrow$ ADRES profil Qualité. Si autonomie critique $\rightarrow$ ADRES profil Économie ou WZ-OSEG taux compression élevé.
6. **Mode d'accès** : Si accès séquentiel uniquement $\rightarrow$ WZ-OSEG. Si accès aléatoire requis $\rightarrow$ ADRES.

Perspectives de convergence hybride

Une approche hybride combinant les forces de WZ-OSEG et ADRES pourrait offrir une solution optimale adaptative :

Architecture hybride proposée :

- Utiliser ADRES pour les images clés (I-frames) transmises périodiquement (par exemple toutes les 10 images)
- Utiliser WZ-OSEG pour les images inter (P-frames) exploitant la corrélation avec l'image clé précédente
- Déclencher dynamiquement une I-frame ADRES lors de la détection d'un changement de scène majeur (indicateurs globaux au-delà d'un seuil)

Cette structure GOP (Group Of Pictures) adaptative combinerait :

- L'efficacité de compression de WZ-OSEG pour les séquences statiques
- La robustesse et l'accès aléatoire d'ADRES via les images clés





- Une récupération automatique après pertes de paquets (resynchronisation à la prochaine I-frame)

L'implémentation d'un tel système hybride constitue une perspective de recherche prometteuse pour maximiser simultanément l'efficacité de compression, la robustesse aux erreurs et la flexibilité d'accès.

2.1.4 Conclusion du chapitre

Ce chapitre a présenté deux algorithmes originaux de compression asymétrique pour réseaux de capteurs multimédias en agriculture de précision : WZ-OSeg et ADRES. Les deux approches exploitent l'asymétrie ressources entre nœuds capteurs contraints et station de base puissante, mais selon des paradigmes distincts.

WZ-OSeg s'appuie sur le codage distribué de sources (théorèmes de Slepian-Wolf et Wyner-Ziv) pour déporter la complexité de décodage vers le puits, tout en intégrant une segmentation hybride Otsu-Hue pour identifier les régions d'intérêt végétales. L'exploitation de la corrélation temporelle via la side information permet d'atteindre des taux de compression élevés (facteur 60) pour les scènes quasi-statiques typiques de l'agriculture.

ADRES adopte une approche spatiale par dégradation régionale différenciée, allouant sélectivement la bande passante selon l'importance sémantique des zones (végétation vs fond). Son indépendance temporelle garantit la robustesse aux pertes de paquets et facilite l'accès aléatoire, tandis que les profils adaptatifs offrent un contrôle fin du compromis qualité/autonomie.

L'analyse de complexité a établi que les deux algorithmes présentent une complexité temporelle linéaire $O(n)$ au nœud capteur, les rendant compatibles avec les microcontrôleurs embarqués modernes. Les volumes de transmission atteints (5 à 20 Ko pour 640×480 pixels) représentent des réductions d'un facteur 45 à 180 par rapport aux images brutes, se traduisant par une prolongation proportionnelle de l'autonomie énergétique.

La validation expérimentale de ces algorithmes sur des datasets d'images agricoles réelles, présentée dans le chapitre suivant, permettra de quantifier précisément leurs performances respectives et de guider le choix de la solution optimale pour le déploiement opérationnel du système de surveillance agricole.